

Flag : `picoCTF{rs4_k3y_1n_1mg_51611ab8}`

📄 Résumé de la faille

Un fichier `flag.enc` est chiffré en RSA, mais la clé publique a "disparu". Une image fournie avec le challenge contient, cachée dans ses métadonnées EXIF (champ Comment), une clé privée RSA encodée en hexadécimal. Une fois extraite et convertie, cette clé permet de déchiffrer directement le fichier avec `openssl`.

Concept : combinaison stéganographie (donnée cachée dans une image) + cryptographie RSA classique.

⚡ Méthodo — les étapes

1. Lire l'énoncé - "The public key is gone... but someone might have been careless with the private key." → indice : la clé privée traîne quelque part, probablement mal protégée. - Hints : "Metadata can tell you more than you expect" + "Hex can be turned back into a key file" → métadonnées + conversion hex.

2. Extraire les métadonnées de l'image

```
exiftool Téléchargements/image.jpg
```

- Résultat : un champ **Comment** contenant une longue chaîne hexadécimale (commençant par `2d2d2d2d2d424547494e...`).

3. Reconnaître le contenu caché - Décodage mental du début : `2d = -`, `42 = B`, `45 = E` ... → `-----BEGIN` - Signature reconnaissable du format **PEM** (clé/certificat).

4. Convertir le hex en texte (CyberChef) - Copier la chaîne hex complète dans CyberChef. - Opération **"From Hex"** (delimiter: Auto) → révèle le bloc complet :

```
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBCcw...
-----END PRIVATE KEY-----
```

5. ⚠️ Piège rencontré : mauvaise première conversion - Une première tentative (passer par "Generate RSA key pair" après un hex mal interprété) a donné une clé RSA **différente et incorrecte** (1024 bits au lieu de la vraie clé). - Symptôme trompeur : `openssl` refusait de déchiffrer avec l'erreur "data greater than mod len" (256 octets de fichier chiffré vs clé de 128 octets max) — laissant croire à un problème de chiffrement par blocs complexe, alors que la cause était juste une **clé mal extraite**. - **Leçon : en cas d'erreur de déchiffrement inattendue, vérifier l'extraction/la conversion en amont avant de chercher une explication plus compliquée du côté chiffrement.**

6. Bonne extraction : **"From Hex" directement sur la sortie exiftool** - En repartant du texte brut donné par exiftool et en appliquant directement **From Hex** (sans étape intermédiaire erronée), on obtient la vraie clé privée RSA, cohérente avec la taille de `flag.enc` (256 octets → clé 2048 bits). - Sauvegarder cette clé dans un fichier : `pico.pem`

7. Déchiffrer avec openssl

```
openssl pkeyutl -decrypt -inkey pico.pem -in flag.enc
```

→ `picoCTF{rs4_k3y_1n_1mg_51611ab8}` 🏁

🔍 Comprendre l'erreur "data greater than mod len"

En RSA, la taille max des données déchiffrables en un bloc est limitée par la taille de la clé (le "module" / modulus) : - Clé 1024 bits → max ~128 octets - Clé 2048 bits → max ~256 octets

Si le fichier chiffré est plus gros que ce que permet la clé fournie, openssl refuse avec cette erreur. Deux causes possibles à distinguer : 1. **La clé utilisée est la mauvaise clé** (notre cas ici — clé mal extraite/convertie) 2. **Le fichier a été chiffré par blocs artisanaux** avec une vraie clé trop petite pour un seul bloc (nécessiterait un script Python avec `pow(c, d, n)` sur chaque bloc)

→ Toujours vérifier l'hypothèse 1 (clé correcte ?) avant de partir sur l'hypothèse 2 (plus complexe à mettre en œuvre).

🧰 Outils utilisés

Outil	Rôle
<code>exiftool</code>	Extraire les métadonnées de l'image (champ Comment)
CyberChef (From Hex)	Convertir la chaîne hex en texte (la clé PEM)
<code>openssl pkeyutl -decrypt</code>	Déchiffrer le fichier avec la clé privée RSA

Commande complète du déchiffrement :

```
openssl pkeyutl -decrypt -inkey pico.pem -in flag.enc
```

- `pkeyutl` : sous-commande openssl pour les opérations à clé publique/privée (remplace `rsautl`, dépréciée depuis OpenSSL 3.0)
- `-decrypt` : mode déchiffrement
- `-inkey pico.pem` : la clé privée à utiliser
- `-in flag.enc` : le fichier chiffré

✅ Réflexes à retenir

- Sur une image/fichier suspect : toujours tester `exiftool` en premier pour les métadonnées cachées.

- Une chaîne hex commençant par `2d2d2d2d2d42454749...` = signature de `-----BEGIN` → probablement une clé/certificat PEM caché.
 - **En cas d'erreur inattendue lors d'un déchiffrement, vérifier d'abord l'extraction/conversion des données en amont** avant de chercher une explication technique plus complexe côté chiffrement.
 - `rsautl` est déprécié depuis OpenSSL 3.0 → utiliser `pkeyutl`.
 - La taille du fichier chiffré doit être cohérente avec la taille de la clé RSA (128 octets pour 1024 bits, 256 pour 2048 bits...) — une incohérence est un signal d'alerte utile pour diagnostiquer.
-

🔗 Concept : stéganographie + RSA

Ce challenge combine deux domaines : - **Stéganographie** : cacher de l'information dans un support qui n'a pas vocation à la contenir (ici, les métadonnées d'une image plutôt que les pixels eux-mêmes — une forme "légère" de stégano). - **RSA** : chiffrement asymétrique classique, où la clé privée permet de déchiffrer ce que la clé publique a chiffré.

Dans la vraie vie : une clé privée ne doit **jamais** être stockée dans des métadonnées, un fichier public, ou tout endroit accessible — c'est exactement la faille que ce challenge illustre ("someone might have been careless with the private key").